



Pandas գրադարան

ԴԱՍ 22



Ընդհանուր բնութագիր

Pandas-ը բաց և հասանելի գրադարան է Python-ում, որն առավել լայնորեն օգտագործվում է տվյալների վերլուծության և մեքենայական ուսուցման մեջ: Pandas-ը կառուցված է Python-ի երկու մեծ գրադարանների հիման վրա՝ matplotlib և NumPy: Pandas-ը համարվում է այս գրադարանների կոմբինացիան, որը թույլ է տալիս օգտագործել matplotlib-ի և NumPy-ի շատ մեթոդներ՝ ավելի քիչ կոդով:





Pandas-ը հեշտացնում է տվյալների հետ աշխատելու շատ ժամանակատար և կրկնվող առաջադրանքներ, այդ թվում՝

- **Տվյալների մաքրում**
 - **Տվյալների լրացում**
 - **Տվյալների նորմալացում**
 - **Միաձուլում և միացում**
 - **Տվյալների վիզուալիզացիա**
 - **Վիճակագրական վերլուծություն**
 - **Տվյալների ստուգում**
 - **Տվյալների բեռնում և պահպանում**
- 



Pandas-ը տվյալների պահպանման համար ներկայացրել է երկու նոր տեսակի օբյեկտներ, որոնք հեշտացնում են վերլուծական առաջադրանքների կատարումն ու վերացնում գործիքները փոխելու անհրաժեշտությունը: Դրանցից առաջինը կոչվում է Series և ունի ցուցակի կառուցվածք, իսկ երկրորդը՝ DataFrames-ն է, որն ունի աղյուսակային կառուցվածք:



Series – սա մեկ չափանի կառուցվածք է, որը կարելի է համարել Python-ի ցուցակ կամ NumPy զանգված: Այն ունի մեկ սյունակ, որտեղ յուրաքանչյուր արժեքին համապատասխանում է իր ինդեքսը:

DataFrame – սա աղյուսակային կառուցվածք է, որը բաղկացած է մի քանի Series-ներից: Այն նման է Excel-ի աղյուսակի կամ SQL-ի աղյուսակի, որտեղ յուրաքանչյուր սյունակ՝ Series է, իսկ տողերը՝ ինդեքսավորված են:

```
import pandas as pd
data = pd.Series([10, 20, 30, 40, 50])
print(data)
```

```
0    10
1    20
2    30
3    40
4    50
dtype: int64
```

```
import pandas as pd
data = pd.DataFrame({
    "Անուն": ["Արմեն", "Աննա", "Դավիթ"],
    "Տարիք": [25, 30, 22],
    "Բնակավայր": ["Երևան", "Գյումրի", "Վանաձոր"]
})
print(data)
```

	Անուն	Տարիք	Բնակավայր
0	Արմեն	25	Երևան
1	Աննա	30	Գյումրի
2	Դավիթ	22	Վանաձոր

DataFrame-ի ստեղծում ձեռքով մուտքագրված տվյալներից

Սա կարող է օգտակար լինել, երբ դուք պետք է ձեռքով մուտքագրեք պարզ տվյալներ ծրագրի մեջ:

```
df = pd.DataFrame([[1, 'Bob', 'Builder'],  
                  [2, 'Sally', 'Baker'],  
                  [3, 'Scott', 'Candle Stick Maker']],  
                  columns=['id', 'name', 'occupation'])
```

	id	name	occupation
0	1	Bob	Builder
1	2	Sally	Baker
2	3	Scott	Candle Stick Maker

DataFrame-ի պատճենում

DataFrame-ների պատճենումը կարող է օգտակար լինել այն իրավիճակներում, երբ դուք պետք է փոփոխություններ կատարեք տվյալների մեջ, բայց պետք է նաև պահպանել բնօրինակը: Եթե DataFrame-ները պետք է պատճենվեն, խորհուրդ է տրվում դա անել դրանք ներբեռնելուց անմիջապես հետո:

```
data_copy = data.copy(deep=True)
```

`deep=True` `deep copy` է ստեղծում, այսինքն՝ տվյալների ամբողջական պատճեն, ներառյալ օբյեկտների մեջ պարունակվող բոլոր տարրերը:

Տվյալների բեռնում

CSV տվյալների ներբեռնում

Pandas-ը հնարավորություն է տալիս աշխատել csv ֆայլերի հետ: csv ֆայլի տվյալները կարող ենք ստանալ `df = pd.read_csv("data.csv")` օգտագործելով `pd.read_csv` հատկանիշը, իսկ չափերտների մեջ՝ ֆայլի հասցեն: Երբ DataFrame-ը պարունակում է անընթեռնելի նշաններ, ապա տվյալները բեռնելիս կարող եք օգտագործել `encoding='ISO-8859-1'` կոդսավորումը: Նման ձևով տվյալներ կարող եք բեռնել նաև Excel ֆայլերից օգտագործելով `pd.read_excel` հրահանգը:

Տվյալների Պահպանում

Ամբողջական տվյալներից ինչ որ հատված պահելիս դրանք պահվորմ են նույն միջավայրում, ինչ ձեր կոդը:

```
rating[:10].to_csv('saved_ratings.csv', index=False)
```

rating[:10] – Վերցնում է առաջին 10 տողը,

.to_csv('saved_ratings.csv', index=False) – Արդյունքը պահպանում է CSV ֆայլում: **.to_csv()** մեթոդը DataFrame-ը պահպանում է CSV ֆայլի տեսքով:

'saved_ratings.csv' – ֆայլի անունն է, որտեղ պետք է պահպանվի տվյալները:

index=False – նշանակում է, որ CSV ֆայլում չի պահպանվելու pandas-ի ինդեքսի սյունակը (քանի որ pandas ինքնաբերաբար ինդեքսավորում է տողերը):

Մեթոդներ տվյալների բեռնման և պահպանման համար

<code>pd.read_csv('file.csv')</code>	Բեռնում է CSV ֆայլ
<code>pd.read_excel('file.xlsx')</code>	Բեռնում է Excel ֆայլ
<code>pd.read_json('file.json')</code>	Բեռնում է JSON ֆայլ
<code>df.to_csv('file.csv', index=False)</code>	Պահպանում է DataFrame-ը CSV ձևաչափով
<code>df.to_excel('file.xlsx', index=False)</code>	Պահպանում է Excel ֆայլում
<code>df.to_json('file.json')</code>	Պահպանում է JSON ֆայլում

DataFrame-ի սկզբից կամ վերջից n գրանցումների ստացում

Կոդի մեջ ինչ որ տեղ կարելի է տալ DataFrame-ում եղած տվյալների առաջին կամ վերջին տողերը, վերհիշելու համար, թե ինչ կա պահված մեր DataFrame-ում: Այդ նպատակով օգտագործվում են `data.head(n)` առաջին n անդամը տալելու, և `data.tail(n)` վերջին n անդամը տալելու համար:

DataFrame-ում տողերի քանակի հաշվում

`len()` ֆունկցիան ներառված չէ Pandas-ի մեջ, բայց այն հարմար է DataFrame-ում տողերի քանակը արագ հաշվելու համար: `len(df)`

Սյունակում չկրկնվող արժեքների քանակի հաշվում

```
len(ratings['user_id'].unique())
```

DataFrame-ի մասին տեղեկատվության ստացում

DataFrame-ի տեղեկատվությունը ներառում է նրա մասին ընդհանուր տեղեկություններ, ինչպիսիք են վերնագիրը, արժեքների քանակը, սյունակի տվյալների տեսակները: **df.info()**

Կա df.info-ի նման մեկ այլ ֆունկցիա՝ **df.dtypes()**: Այն ցուցադրում է միայն տեղեկատվություն սյունակների տվյալների տիպերի մասին:

DataFrame-ի մասին վիճակագրական տեղեկատվության ստացում և ցուցադրում

DataFrame-ի մասին վիճակագրական տեղեկատվության ստացումը շատ օգտակար է այն իրավիճակներում, երբ DataFrame-ը պարունակում է բազմաթիվ թվային արժեքներ: **df.describe()**

Մեթոդներ տվյալների հետազոտության համար

<code>df.head(n)</code>	Ցույց է տալիս առաջին n տողերը (դեֆոլտ՝ 5)
<code>df.tail(n)</code>	Ցույց է տալիս վերջին n տողերը (դեֆոլտ՝ 5)
<code>df.shape</code>	Վերադարձնում է (տողերի քանակ, սյունակների քանակ)
<code>df.info()</code>	Ցուցադրում է DataFrame-ի ընդհանուր ինֆորմացիան
<code>df.describe()</code>	Վիճակագրական ամփոփ տվյալներ (միջին, մին., մաքս.)

Տվյալների Ինդեքսավորում

<code>df[column_name]</code>	Վերադարձնում է տվյալ սյունակը (Series)
<code>df.iloc[row_index, column_index]</code>	Ինդեքսավորում ըստ թվային ինդեքսների
<code>df.loc[row_label, column_label]</code>	Ինդեքսավորում ըստ տողի և սյունակի անունների
<code>df.drop(labels, axis, inplace)</code>	Ջնջում է նշված տողերը կամ սյունակները, <code>labels</code> (Ջնջվող տողերի կամ սյունակների անունները), <code>axis=0</code> (Ջնջում է տողերը (default), <code>axis=1</code> (Ջնջում է սյունակները)
<code>df.set_index(column)</code>	Ինդեքսի սյունակը փոխում է մեր ուզած ձևով
<code>df.reset_index()</code>	Վերադաձնում է նախկին ինդեքսավորմանը

```
import pandas as pd

data = {
    "Անուն": ["Աննա", "Արմեն", "Լիլիթ", "Դավիթ"],
    "Տարիք": [25, 30, 22, 28],
    "Գնահատական": [90, 85, 88, 92]}

```

```
df = pd.DataFrame(data)

```

```
# Փոխենք "Արմեն"-ի տարիքը 35-ի

```

```
df.loc[1, "Տարիք"] = 35

```

```
print(df)

```

loc[1, "Տարիք"] = 35 նշանակում է՝ 1-ին տողի "Տարիք" սյունակում գտնվող արժեքը դարձնել 35:

```
# Փոխենք "Արմեն"-ի տարիքը 35-ի

```

```
df.loc[df["Անուն"]=="Արմեն", "Տարիք"] = 35

```

```
print(df)

```

Փնտրում է Արմեն-ը Անուն սյունակում և փոխում տարիքը 35

```
# Փոխենք 2-րդ տողի 3-րդ սյունակի արժեքը (Լիլիթի գնահատականը)
df.iloc[2, 2] = 95
print(df)

```

iloc[2, 2] = 95 նշանակում է՝ 2-րդ տողի 3-րդ սյունակի(Գնահատական) արժեքը դարձնել 95:

```
# Փոխենք 1-ին և 2-րդ տողերի "Գնահատական"-ները

```

```
df.loc[[1, 2], "Գնահատական"] = [75, 82]

```

```
print(df)

```

Այս փոփոխությունը Արմեն-ի գնահատականը կդարձնի 75, իսկ Լիլիթ-ինը՝ 82

```
# Բոլոր "Գնահատական"-ները դարձնենք 95

```

```
df["Գնահատական"] = 95

```

```
print(df)

```

Այս եղանակով բոլոր "Գնահատական"-ները ստանում են 95 արժեք: